ENGR1050: Lecture 19

Review to prepare for Exam 2

November 2, 2026



Agenda

Today we will prepare for the exam by reviewing the material since Exam 1, including:

- Numpy basics
- Definition of ODEs and initial value problems
- Numerical solution of ODEs using Explicit Euler
- Usage of the ODE_solver class and inheritance to create new ODE solvers
- Programming the Pico to read a potentiometer and wire buttons/LEDs
- Reasoning about numerical solutions: refinement and steady state behavior

I'll run through the material you should be comfortable with in these slides, and then we will work through some examples in the Jupyter notebook.

Exam 2 format

- The exam will be **in person** during our normal class time next lecture.
- All necessary code and syntax will be provided (no need to memorize).
- No notes are allowed. As in Exam 1, a cheat sheet will be provided with the exam.
- You'll need to show your Penn ID.
- Similar format to last Exam. You should anticipate:
  - Reading some code and explaining what it does
  - Writing some simple code that makes it clear you understand code from HWs and labs
  - **New:** Perform some mathematical calculations related to ODEs

You can count on having access to the ODE_solver, button, and LED classes during the exam.

Review: Ordinary Differential Equations (ODEs)

To check that you understand what it means to solve an ODE, you should be comfortable with the following:

- Definition of a kth order ODE: An equation involving an unknown function $y(t)$ and its derivatives up to order k.
- Definition of a initial value problem: **A kth-order ODE requires k initial conditions** to specify a unique solution along with satisfying the ODE itself.
- Check that a given function is a solution by (1) substituting it into the ODE and (2) verifying the initial conditions.
- Rewrite a kth-order ODE as a system of k first-order ODEs by introducing new variables for each derivative,

$$\dot{y_1} = y_2, \quad \dot{y_2} = y_3, \quad \ldots, \quad \dot{y_k} = f(t, y_1, y_2, \ldots, y_k).$$

- Analyze steady state behavior of simple ODEs by setting derivatives to zero and solving for equilibrium values for variables.

Example: Damped harmonic oscillator

The damped harmonic oscillator is governed by the second-order ODE

$$my''(t) + cy'(t) + ky(t) = 0,$$

where m is mass, c is damping coefficient, and k is spring constant.

- To check if $y(t) = e^{-\frac{c}{2m}t} \cos\left(\sqrt{\frac{k}{m} - \frac{c^2}{4m^2}}\, t\right)$ is a solution, we would compute $y'(t)$ and $y''(t)$, substitute into the ODE, and verify that it satisfies **two** initial conditions.
- To rewrite as a system of first-order ODEs, define $v(t) = y'(t)$:

$$\begin{cases} y' = v, \\ v' = -\frac{c}{m}v - \frac{k}{m}y. \end{cases}$$

- To find steady state behavior, set $y' = 0$ and $v' = 0$:

$$\begin{cases} 0 = v_{ss}, \\ 0 = -\frac{c}{m}v_{ss} - \frac{k}{m}y_{ss}. \end{cases}$$

  This implies $v_{ss} = 0$ and $y_{ss} = 0$ at steady state.

Remember!

- In general, ODEs are challenging to solve analytically.
- You will learn how to solve ODEs by hand in a later class.
- We solve ODEs numerically in this class using Explicit Euler.
- That's why we *confirm* that a given solution satisfies the ODE and ICs, and then confirm that our code works by reproducing a known solution.

Review: Explicit Euler method

Remember that our main tool for numerically solving ODEs is the Explicit Euler method.

Definition

For the IVP $y'(t) = f(t, y)$ with $y(t_0) = y_0$ and step size $h > 0$, starting from (t_0, y_0), the Explicit Euler scheme updates via

$$t_{n+1} = t_n + h, \qquad y_{n+1} = y_n + h\, f(t_n, y_n).$$

You should be comfortable using explicit Euler in two ways:

1. Writing a for loop that will implement the method for a simple ODE. (e.g., $y' = -2y$ with $y(0) = 1$.)
2. Using our ODE_solver class to implement the method using class inheritance

You may be given the following ODE_solver class

```python
class ODE_solver:
    def __init__(self, t0, tf, y0, N):
        self.y0 = y0  # initial condition as numpy array
        self.t0 = t0  # initial time as float
        self.tf = tf  # final time as float
        self.N = N    # number of steps as integer
        self.h = (tf - t0) / float(N) # timestep size

        # We need to check what size y0 is and store that information
        self.dim = len(y0) #assumes that y0 is a list or array

        # Create a place to store the solution
        self.t_values = np.array([t0 + i*self.h for i in range(N+1)])  # time values
        self.solution = np.zeros((N+1, self.dim))  # solution array
        self.solution[0, :] = y0  # set initial condition

    def f(self,t,y):
        # Define the right-hand side of the ODE here, assuming a scalar t and a numpy array y
        # Example: dy/dt = -2y (for scalar y),
        # this function should return a numpy array of the same shape as y
        f = np.zeros_like(y)
        f[0] = -2 * y[0]  # Example for the first component
        return f

    def solve(self):
        # Implement explicit euler: y_n+1 = y_n + h*f(t_n, y_n)
        for n in range(self.N):
            t_n = self.t_values[n]
            y_n = self.solution[n, :]
            f_n = self.f(t_n, y_n)
            self.solution[n+1, :] = y_n + self.h * f_n
        return self.solution
```

You must be able to use inheritance to solve new ODEs

You will be provided with an example like this:

```
class childODE_solver(ODE_solver):
      def __init__(self, t0, tf, y0, N, newParameter):
        super().__init__(t0, tf, y0, N)
        self.newParameter = newParameter
    def f(self,t,y):
        # Override the f method to define a new ODE
        f = np.zeros_like(y)
        f[0] = -self.newParameter * y[0]  # Example for the first component
        return f
# Example usage:
t0 = 0.0
tf = 1.0
y0 = np.array([1.0])
N = 100
solver = childODE_solver(t0, tf, y0, N, newParameter=3.0)
solution = solver.solve()
```

You should be comfortable:

- Modifying the `f` method to implement different ODEs.
- Changing the initial conditions when creating an instance of the child class.
- Understand the shape of `y0` and how it relates to the order of the ODE being solved.

Trusting your simulation

To make sure our code is properly configured, we always verify that our numerical solution is reasonable. Understand the logic behind these techniques:

- **Compare to analytical solutions:** Whenever possible, compare your numerical results to known analytical solutions for simple cases.
- **Refinement study:** Run simulations with decreasing step sizes h and observe if the solution converges.
- **Steady state analysis:** Check if the numerical solution approaches the expected steady state as t increases.

Understanding the labs/Pico programming

Finally, make sure you understand the key concepts from the labs involving the Raspberry Pi Pico:

- How the initialization of pins maps onto the wiring of the actual device.
- The inputs and outputs of LEDs, buttons, and potentiometers.
- How to use the simple classes we provided for buttons and LEDs.
- The logic behind setting up the potentiometer: what is ADC, what pins do what, and how to calibrate readings to meaningful angle measurements.

You may be given the following LED class

```python
from machine import Pin
import time

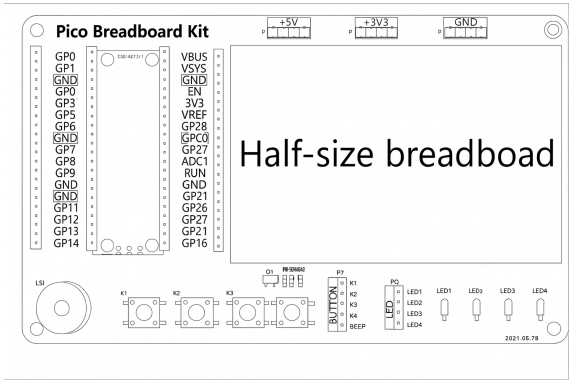
class LEDClass:
    def __init__(self, pin_number, initial_state=False):
        """
        Initialize LED class
        pin_number: GPIO pin number for the LED
        initial_state: True for on, False for off (default: False)
        """
        self.led = Pin(pin_number, Pin.OUT)
        self.state = initial_state

        # Set initial state
        if self.state:
            self.led.on()
        else:
            self.led.off()

    def toggle(self):
        """Toggle the LED state (on->off or off->on)"""
        if self.state:
            self.led.off()
            self.state = False
        else:
            self.led.on()
            self.state = True
```

Example: Wiring up a button/LED

Given a diagram, you should be able to:



- Given a wiring diagram, write a constructor for a button/LED class that correctly matches the wiring.
- Given an initialization, sketch on the diagram what pins need to be connected to which components.

Example: Initialize an LED on pin 2 initially on.

```
led = LEDClass(pin_number=2, initial_state=True)
```

Any more questions for the exam?

Closing note: Using scipy.integrate.odeint

While we've focused on our custom ODE_solver class, Python's `scipy` library provides a powerful built-in function called `odeint`.

```
from scipy.integrate import odeint
import numpy as np
import matplotlib.pyplot as plt

# Define the ODE: dy/dt = f(y, t)
def f(y, t):
    return -2 * y  # Example: dy/dt = -2y

# Initial condition and time points
y0 = 1.0
t = np.linspace(0, 2, 100)

# Solve the ODE
solution = odeint(f, y0, t)

# Plot results
plt.plot(t, solution)
```

Advanced ODE Solvers: Beyond Explicit Euler

While Explicit Euler is simple enough for us to implement and understand, in the real world we use more advanced methods.

Higher-Order Methods (Better Accuracy):

- **Runge-Kutta 4th order (RK4):** Uses 4 function evaluations per step for much better accuracy
- **Implicit methods:** Better stability for "stiff" ODEs (fast and slow dynamics)
- **Multistep methods:** Use information from previous steps to improve accuracy

Adaptive Methods (Smart Step Size):

- **Error estimation:** Compare solutions with different step sizes to estimate error
- **Automatic step control:** Increase h in smooth regions, decrease h near rapid changes
- **Tolerance-based:** User specifies desired accuracy, solver adjusts h automatically

Today's exercise: Simplified pandemic modeling

In today's exercise, we will apply what we've learned to model the spread of a pandemic using a system of ODEs.

SIR Model:

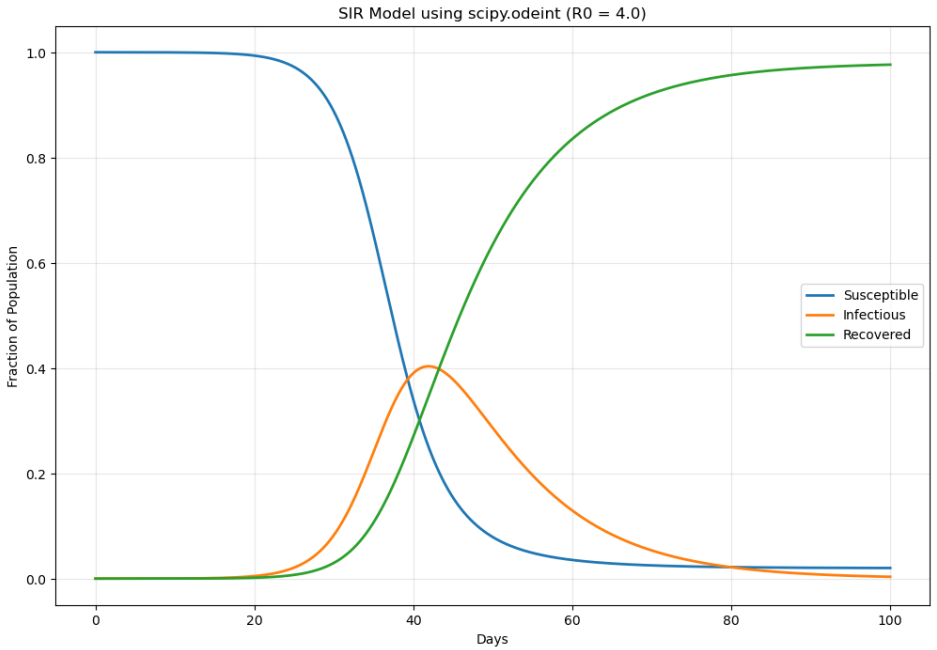
- **S:** Susceptible individuals
- **I:** Infected individuals
- **R:** Recovered individuals

The SIR model is governed by the following ODEs:

$$\begin{aligned}\frac{dS}{dt} &= -\frac{\beta}{N}SI,\\ \frac{dI}{dt} &= \frac{\beta}{N}SI - \gamma I,\\ \frac{dR}{dt} &= \gamma I,\end{aligned}$$

where β is the transmission rate and γ is the recovery rate, and $N = S + I + R$ is the total population.

Representative solution



$$\frac{dS}{dt} = -\beta SI,$$
$$\frac{dI}{dt} = \beta SI - \gamma I,$$
$$\frac{dR}{dt} = \gamma I,$$